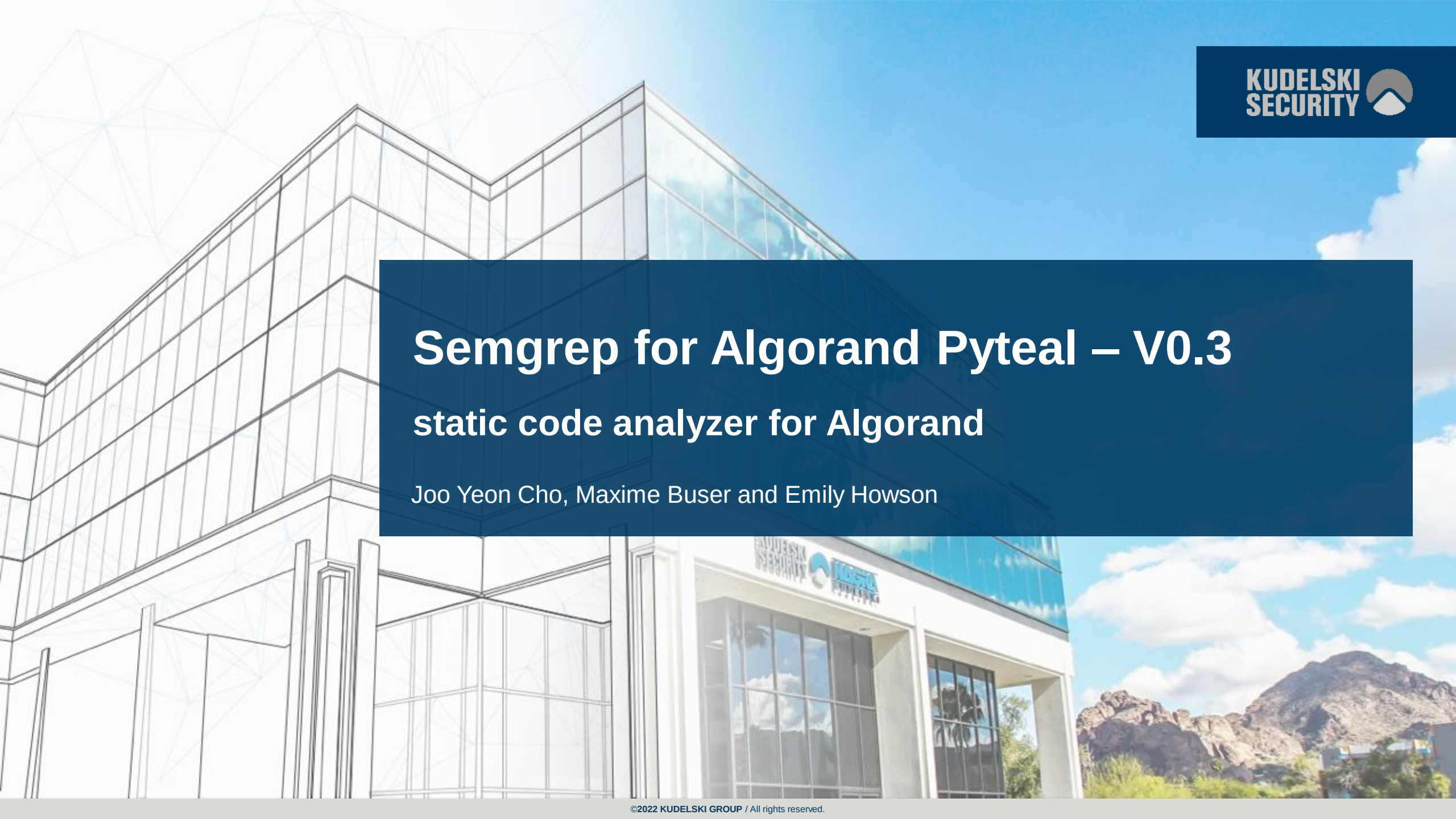


KUDELSKI SECURITY

Semgrep for Algorand Pyteal – V0.3

static code analyzer for Algorand

Joo Yeon Cho, Maxime Buser and Emily Howson

©2022 KUDELSKI GROUP / All rights reserved.

Motivation

- Static analysis for smart contract
  - (some of) Vulnerability detection
  - Code audit assistance

    => helping detect hotspots for further manual investigation.

- Compared to Ethereum, Algorand lacks a static code analyzer.
  - For Solidity: Echidna, Slither, Manticore (trail of bits), Securify (Eth foundation), Mythril, MythX, Octopus, MadMax, Gigahorse,…
  - For Pyteal/Teal: Pylint, Tealer

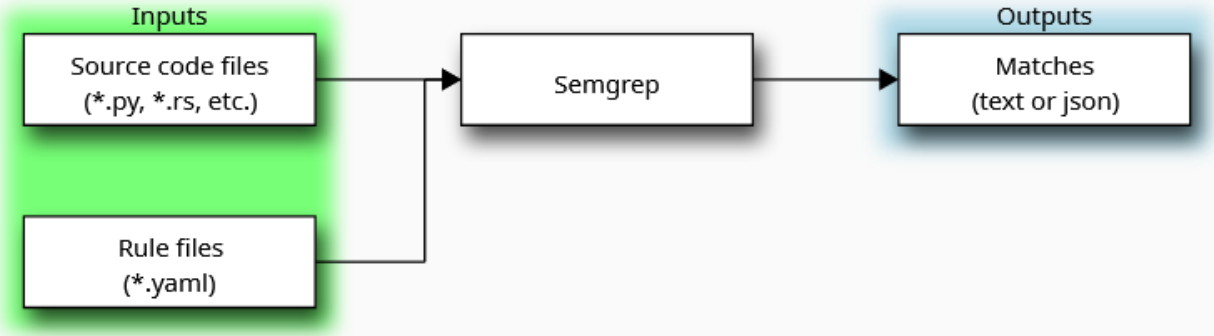
Can we use Semgrep for Algorand Pyteal code analysis?

KUDELSKI SECURITY

©2022 KUDELSKI GROUP / All rights reserved.

Semgrep

- Open-source static analysis tool made by r2c (return2corp)
  - https://semgrep.dev/



(By Nils Amiet)

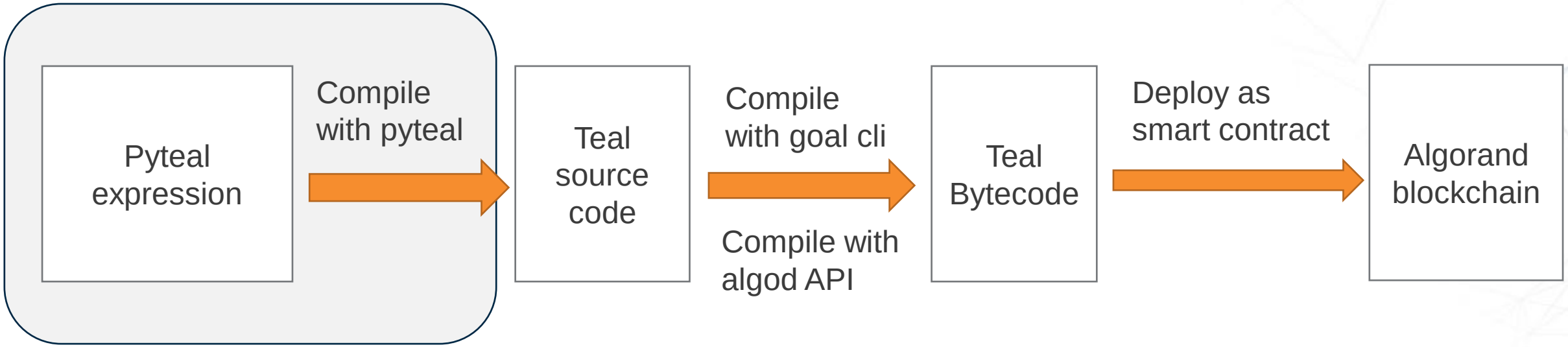
GA ✅	Beta 🐛	Experimental 🚧
C#	Kotlin	Bash
Go	Terraform	C
Java		C++
JavaScript		Dockerfile
JSON		HTML
JSX		Lua
PHP		OCaml
Python		R
Ruby		Rust
Scala		Solidity
TSX		YAML
TypeScript		Generic (ERB, Jinja, etc.)

KUDELSKI SECURITY

©2022 KUDELSKI GROUP / All rights reserved.

PyTeal

- Python library for writing algorand smart contract
- But, pyteal != python



KUDELSKI SECURITY

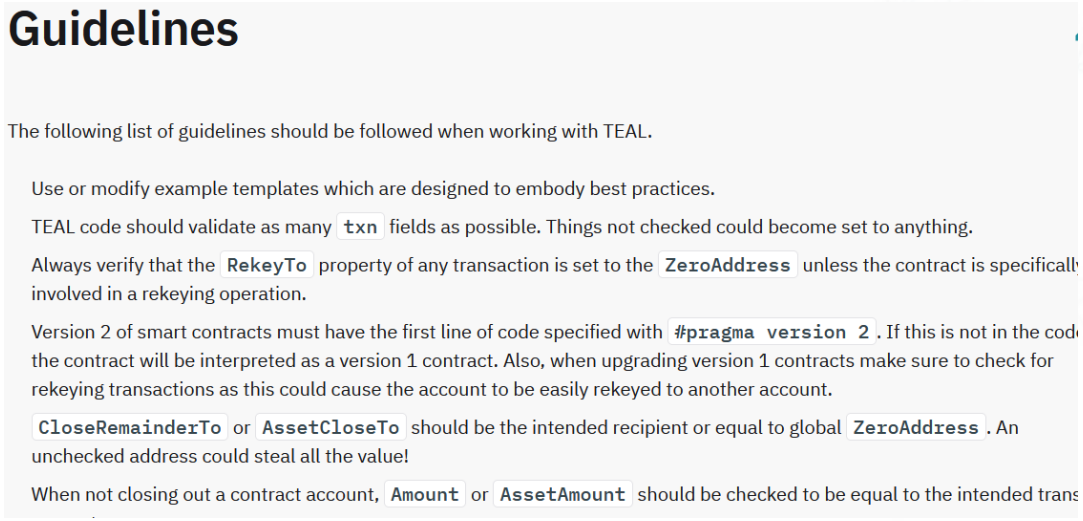
©2022 KUDELSKI GROUP / All rights reserved.

Algorand Semgrep Development - Wecome to contribute!

- Under construct: https://gitlab.kudelski.com/emea-blockchain/algorand-semgrep/

- Currently, 9 rules are supported.
  - Guideline from Algorand dev doc
  - Tinyman hack

- In future, we will add
  - Reentrancy
  - Oracle manipulation
  - Overflow, underflow
  - Unchecked return
  - Unchecked timestamp
  - …



Guidelines

The following list of guidelines should be followed when working with TEAL.

Use or modify example templates which are designed to embody best practices.

TEAL code should validate as many `txn` fields as possible. Things not checked could become set to anything.

Always verify that the `RekeyTo` property of any transaction is set to the `ZeroAddress` unless the contract is specifically involved in a rekeying operation.

Version 2 of smart contracts must have the first line of code specified with `#pragma version 2`. If this is not in the code the contract will be interpreted as a version 1 contract. Also, when upgrading version 1 contracts make sure to check for rekeying transactions as this could cause the account to be easily rekeyed to another account.

`CloseRemainderTo` or `AssetCloseTo` should be the intended recipient or equal to global `ZeroAddress`. An unchecked address could steal all the value!

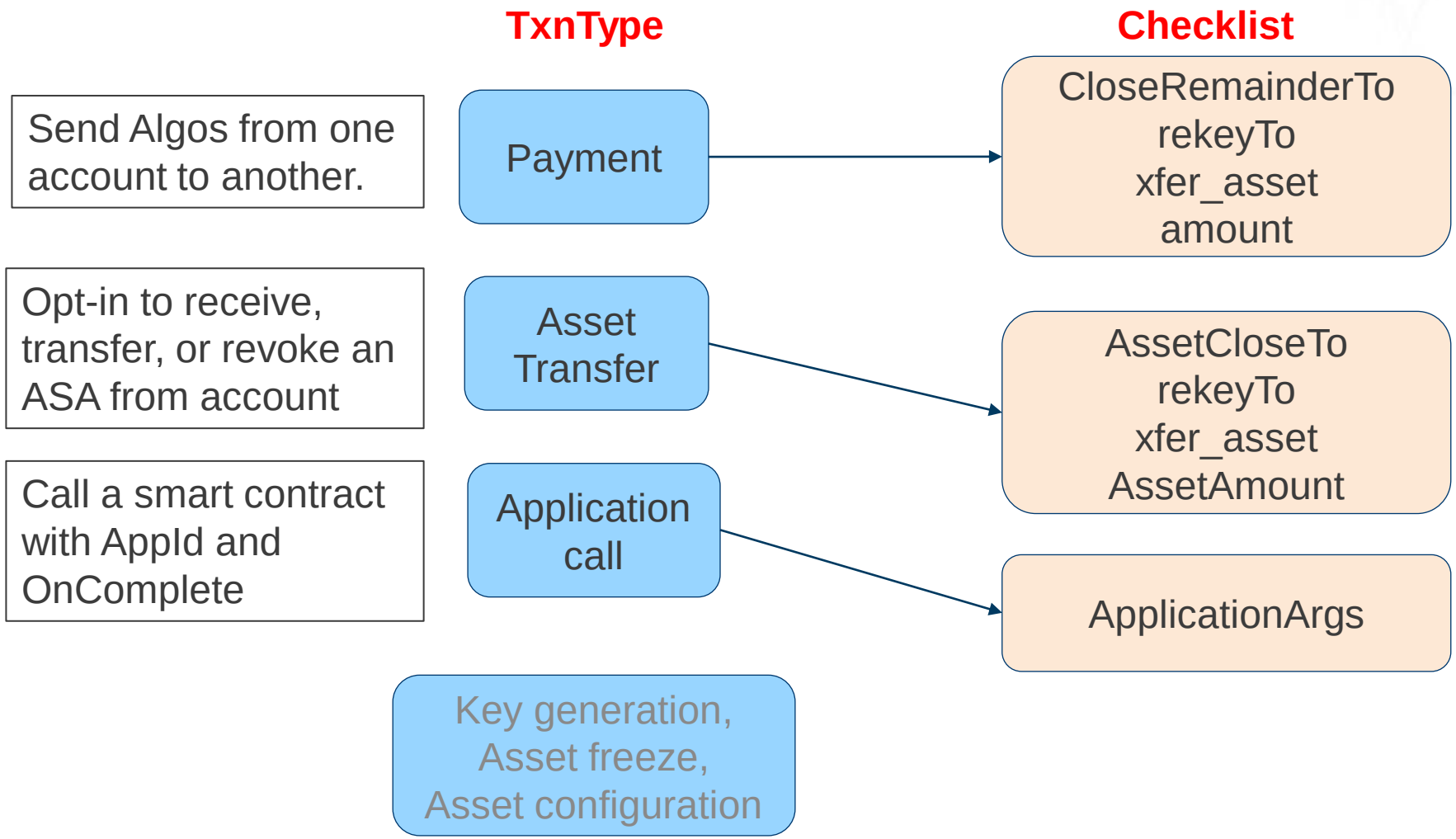
When not closing out a contract account, `Amount` or `AssetAmount` should be checked to be equal to the intended trans

https://developer.algorand.org/docs/get-details/dapps/avm/teal/guidelines/

KUDELSKI SECURITY

©2022 KUDELSKI GROUP / All rights reserved.

How to build rules for DeFi protocol

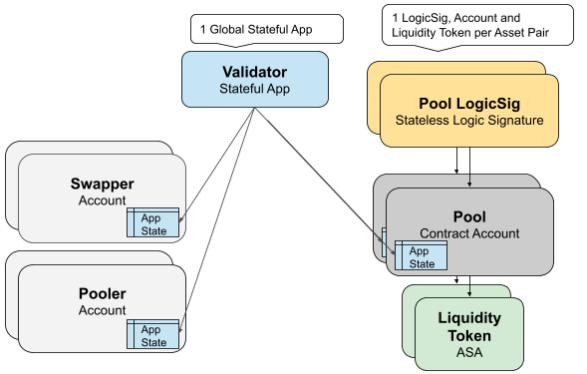


KUDELSKI SECURITY

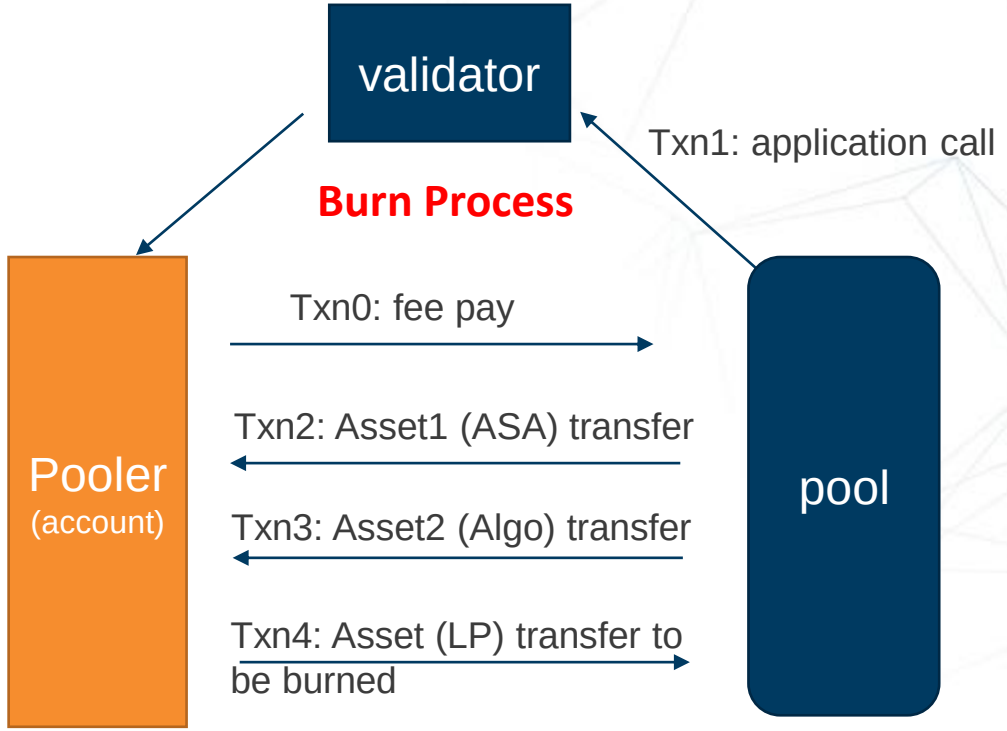
©2022 KUDELSKI GROUP / All rights reserved.

Tinyman Hack – Jan. 2022

- In a burn operation, Tinyman failed to check
  1. Txn3: of the correct type (Algos vs. ASAs)
  2. Txn2 and Txn3: Asset transfers for the expected asset ID



- Attacks
  - Construct a tx group to transfer only ASA for both Txn2 and Txn3 (e.g. 2 goBTC and 600 goBTC)
  - ASA is higher-valued than Algo.
  - Around $3M assets were stolen.



- For details, see the audit report by runtime verification (after attacked)
  https://github.com/runtimeverification/publications/blob/main/reports/smart-contracts/Tinyman-security-review.pdf

KUDELSKI SECURITY

©2022 KUDELSKI GROUP / All rights reserved.

Avoding Tinyman Hack - Asset ID Check

- If TxnType is Payment or AssetTransfer, xfer_asset should be checked.

Semgrep rule

```
patterns:
    - pattern:
        $TXN.type_enum() == TxnType.$TYPE
    - metavariable-pattern:
        metavariable: $TYPE
        pattern-either:
          - pattern: Payment
          - pattern: AssetTransfer

    - pattern-not-inside: |
        <... $TXN.xfer_asset() ...>
```

PyTeal code

```
Gtxn[2].type_enum() ==
TxnType.AssetTransfer,
Gtxn[2].xfer_asset() ==
Int(TMPL_ASSET_ID_1)


Gtxn[3].type_enum() ==
TxnType.Payment,
Gtxn[3]. xfer_asset() ==
Int(TMPL_ASSET_ID_2)
```

Teal code

```
// ensure asset id is asset 1
gtxn 2 XferAsset
int TMPL_ASSET_ID_1
==
assert


// ensure asset id is asset 2
gtxn 3 XferAsset
int 0
gtxn 3 TypeEnum
int pay
== // check if Algo
select
int TMPL_ASSET_ID_2
==
assert
```

KUDELSKI SECURITY

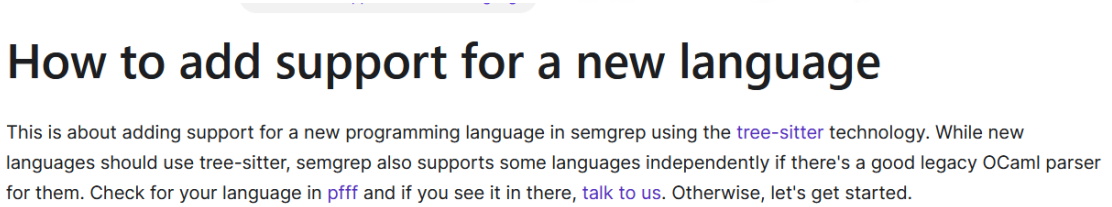
©2022 KUDELSKI GROUP / All rights reserved.

Static Analzer for Teal?

- Tealer (by Trail of Bits): https://github.com/crytic/tealer
  - 4 vulnerabilities detectors and printers allowing to quickly review the contracts.

- Semgrep?
  - Not supporting Teal language.
  - Need to check whether Teal can be added.

- Building a new tool?
  - Need time and fund ☺

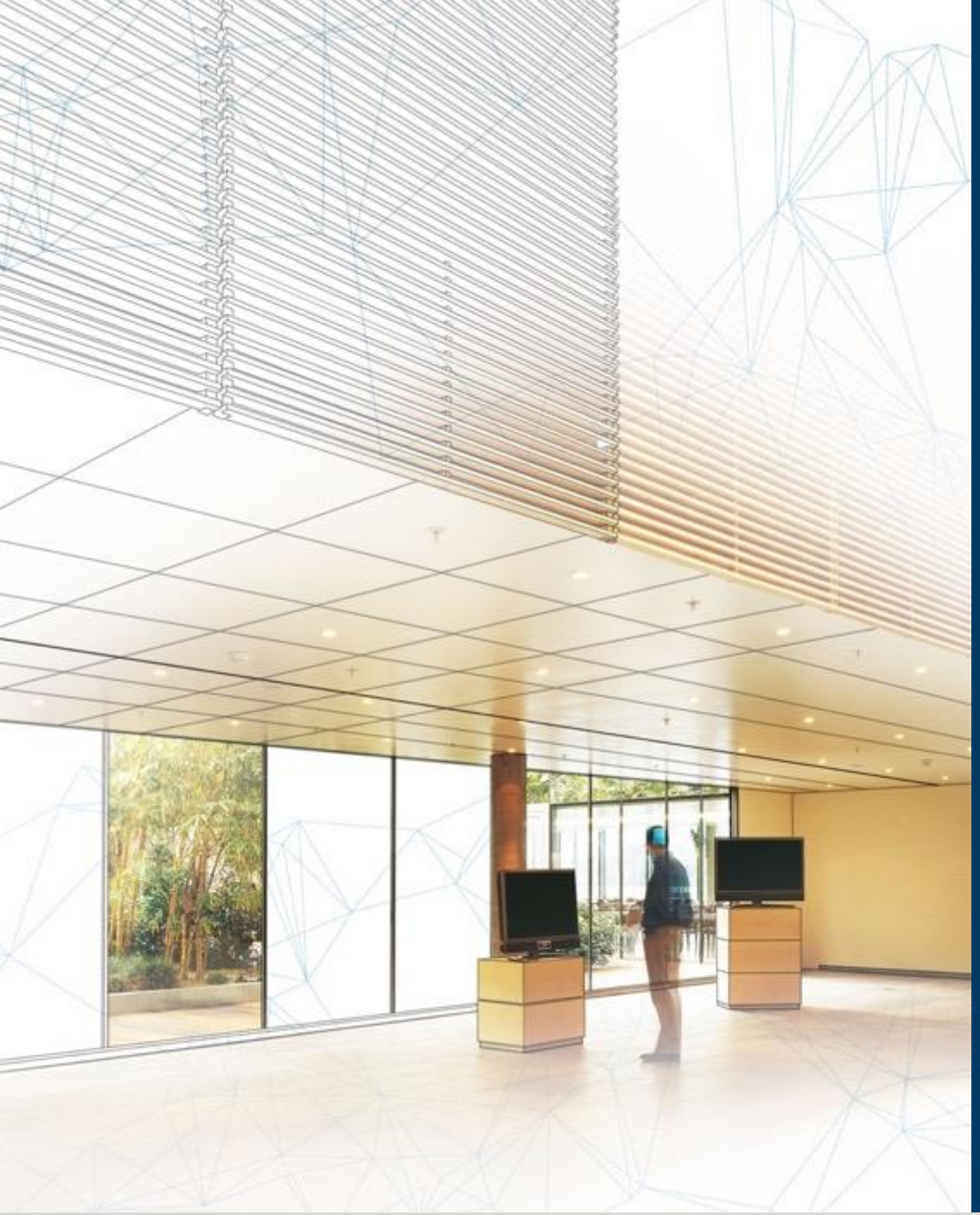


How to add support for a new language

This is about adding support for a new programming language in semgrep using the tree-sitter technology. While new languages should use tree-sitter, semgrep also supports some languages independently if there's a good legacy OCaml parser for them. Check for your language in pfff and if you see it in there, talk to us. Otherwise, let's get started.

KUDELSKI SECURITY

©2022 KUDELSKI GROUP / All rights reserved.



KUDELSKI SECURITY

Thank you

Joo Joo.cho@kudelskisecurity.com

©2022 KUDELSKI GROUP / All rights reserved.